SDN Community Contribution

(This is not an official SAP document)

Disclaimer & Liability Notice

Introduction to JMX Architecture

Applies To:

The Java Management Extensions (JMX) architecture provides interfaces for defining runtime monitoring, management and configuration support for Java applications. The JMX is part of both J2SE 5 and J2EE 1.4 specifications.

Summary

This article provides an introduction to the JMX architecture and briefly describes the key terminologies associated with the JMX technology.

By: Tarun Telang

Company and Title: SAP, Project Intern

Date: 21 April 2005

Table of Contents

Introduction to Java Management Extensions (JMX) Architecture

Introduction

Java Management Extensions (JMX) is an open technology specification that defines the management architecture, which enables management and monitoring of applications and services.

By design, this standard is suitable for adapting legacy systems, implementing new solution for management, and monitoring and plugging into future applications.

JMX is used to provide application management for:

- Web applications.
- Stand-alone applications.
- Application servers.
- Services.
- Java-enabled devices.

The role of JMX in J2EE applications is to:

- Expose portions of JMS and EJB components to a management console.
- Monitor the application server.
- Configure the runtime environment for your application.

JMX Architecture

The JMX architecture has three layers:

- distributed layer
- agent layer
- instrumentation layer

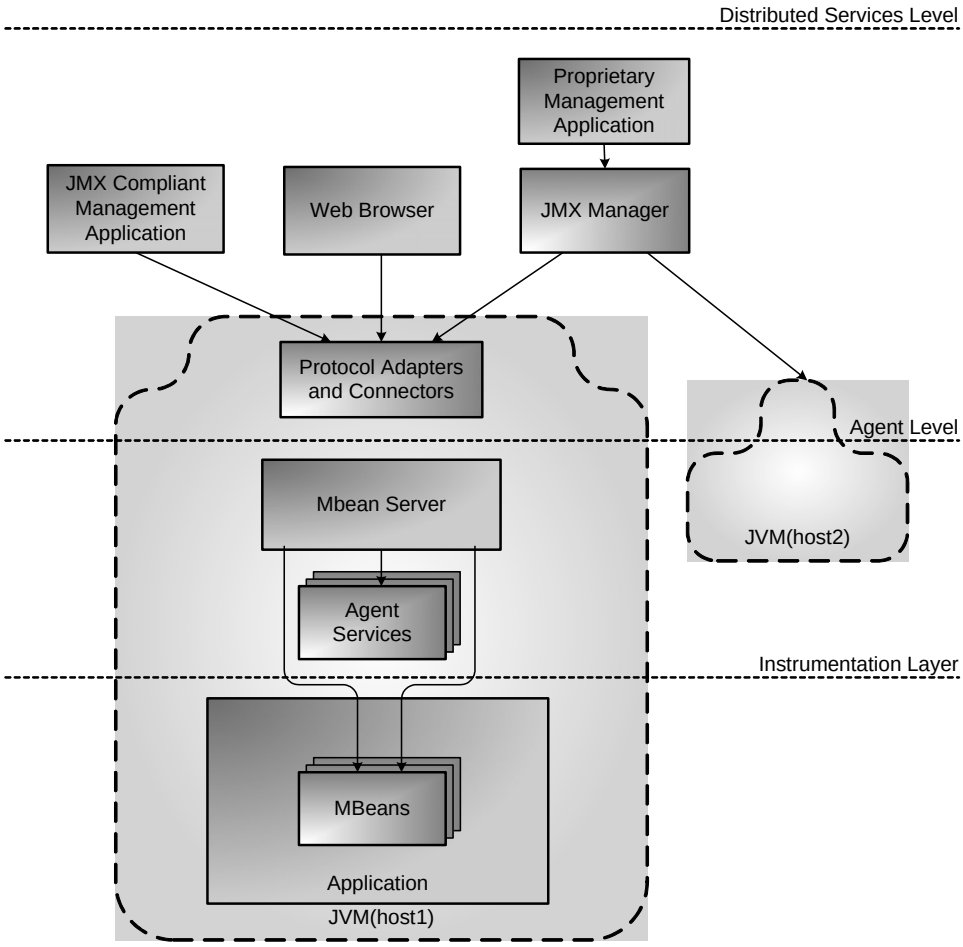

Figure 1: JMX Architectural Overview -The instrumentation level consists of the application and its managed beans. The agent level embodies the JMX services and MBean server. Both the instrumentation and agent level resides within the same JVM. Management applications can remotely access the agent through protocol adaptors and connectors.

Instrumentation Layer

Instruments are the resources to manage. A resource can be one of the following objects:

- Application configuration setting
- Program module
- User identity
- Device

The instrumentation layer contains the MBeans and their manageable resources. You use MBean (metadata classes) from this layer to encapsulate a resource and expose the resource for management.

MBean

An MBean is a named managed object representing a resource. An MBean or "managed bean" is a Java class that meets naming and inheritance standards dictated by JMX. An MBean contains:

- Attributes that can be read and/or written.
- Operations that can be invoked.
- Notifications that the MBean can send.

A standard MBean exposes attributes by using getter and setter methods. MBean objects expose a management interface to a management console. The management interface of a resource is the set of all necessary information and controls that a management application needs to operate on the resource. MBeans are used to send notification in a JMX environment.

Different types of MBeans are:

- Standard MBean
- Dynamic MBean
- Model MBean
- Open MBean

Standard MBean

Standard MBean is the simplest to design and implement. Standard MBeans have static interfaces.

Dynamic MBean

Dynamic MBeans expose their management interface at run-time for maximum flexibility.

Model MBean

Model MBeans are Dynamic MBeans which are configurable at runtime through descriptors using field-value pairs. Model MBean can cache attribute and operation return values for improved performance. Model MBeans automatically send attribute change notifications.

Open MBean

Open MBeans are Dynamic MBeans, and specified to be discovered and used without prior knowledge of types. Open MBeans use predetermined types. Types are revealed through Type Info class.

JMX Notification Model

MBean can both send and receive notifications. Object outside a JMX agent can also receive notifications. There are two types of notifications in JMX:

Notifications at a specific date and time.

Periodic notifications with an optional number of occurrences.

Agent Layer

Agents are the controllers of the instrumentation level objects. JMX agents can be embedded into applications. The agent layer contains the protocol adaptors and connectors. The agent layer also contains the runtime environment for implementing monitoring policies, or the MBean Server.

Agents Services

JMX agents are Java processes that provide a set of services for MBeans. Services are themselves MBeans. JMX agents provide MBeans for the following services:

- Monitoring service
- Relation service
- Timer service
- M-let service (management applet for dynamic loading of MBeans)

Monitoring Services

A monitor observes an attribute of an MBean, the observed attribute, at intervals given as the granularity period. From this observation, a value is derived - the derived gauge. When the derived gauge satisfies one of a set of conditions a notification is issued. There are three kinds of monitors:

- Counter Monitor
- Gauge Monitor
- String Monitor

Relation service

Relational service helps to relate MBeans together as a single unit. It supports associations among MBeans with named roles and cardinality. Consistent cardinality is monitored and when MBeans are unregistered, inconsistent relations are discarded. Relations can also be queried.

Timer service

All listeners to a timer receive all notifications. A timer can be started and stopped, and accumulated notifications are discarded and sent.

Dynamic class loading

Dynamic class loading through m-let (management applet) service retrieves and instantiates new classes and native libraries from an arbitrary network location.

MBean Server

MBean Server is the core component of a JMX agent. It provides a single entry point for calling MBeans in a uniform fashion from management applications in other JVM.

Every MBeans must register their presence with this server so that the management framework can detect them. Thus an MBean Server acts as a registry for MBeans. MBean servers discover interfaces through introspection. The MBean server also handles the management messages that are sent among registered MBeans.

Distributed Layer

The distributed layer is the outermost layer, responsible for exposing JMX agents to management applications. A distributed service is the mechanism by which administration applications interact with agents and their managed objects.

To make JMX agents available, the distributed layer uses adapters or connectors. The management console uses following protocols to communicate with remote JMX agents and their MBeans:

- Java RMI
- Jini network technology
- TCP/IP
- HTTP
- SNMP

Summary

Java Management Extension provides interfaces for adding management and monitoring capabilities to Java applications, services and devices.

JMX is recommended for:

- Java applications that require complex management capabilities.
- Situations in which you need to control as well as monitor your applications.

Advantages of using JMX:

- Works with any J2EE and J2SE applications (part of standard J2SE 5 and J2EE 1.4).
- Affordable to implement. (Instrumentation is easy to implement.)
- Provides detailed application visibility control.
- Provides remote management.
- Provides two-way management.

- Supported industry standard.

Disadvantages of using JMX:

- Requires a JMX server.
- Has had some maturity issues in accessing its interface; however, specification JSR160 is now addressing these.

Disclaimer & Liability Notice

This document may discuss sample coding, which does not include official interfaces and therefore is not supported. Changes made based on this information are not supported and can be overwritten during an upgrade.

SAP will not be held liable for any damages caused by using or misusing of the code and methods suggested here, and anyone using these methods, is doing it under his/her own responsibility.

SAP offers no guarantees and assumes no responsibility or liability of any type with respect to the content of the technical article, including any liability resulting from incompatibility between the content of the technical article and the materials and services offered by SAP. You agree that you will not hold SAP responsible or liable with respect to the content of the Technical Article or seek to do so.

Author Bio

Tarun Telang is working as an intern at SAP Labs, India. Currently he is working on projects based on the SAP NetWeaver platform. Tarun is also pursuing his studies at the Indian Institute of Information Technology, Bangalore (IIITB).